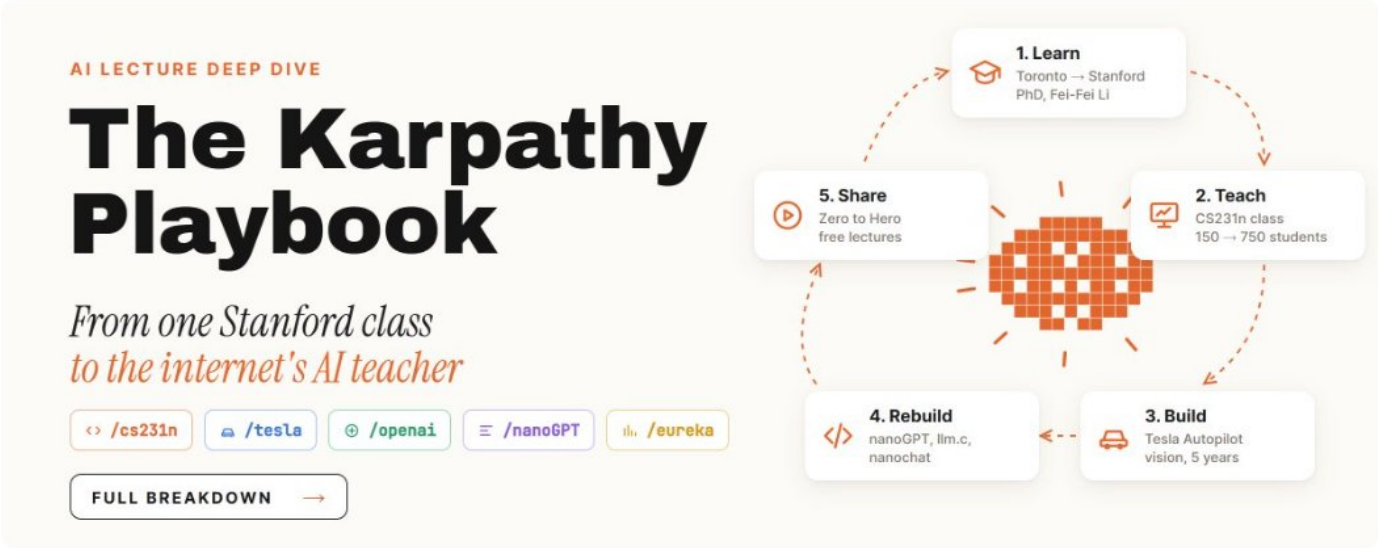
The Karpathy Playbook: How Teaching Became the Most Valuable Skill in AI

Author: Dmitro Cashback (@DmitroCP)
Date: October 25, 2026 at 03:59 PM
Source: https://x.com/DmitroCP/status/2114303402083856499

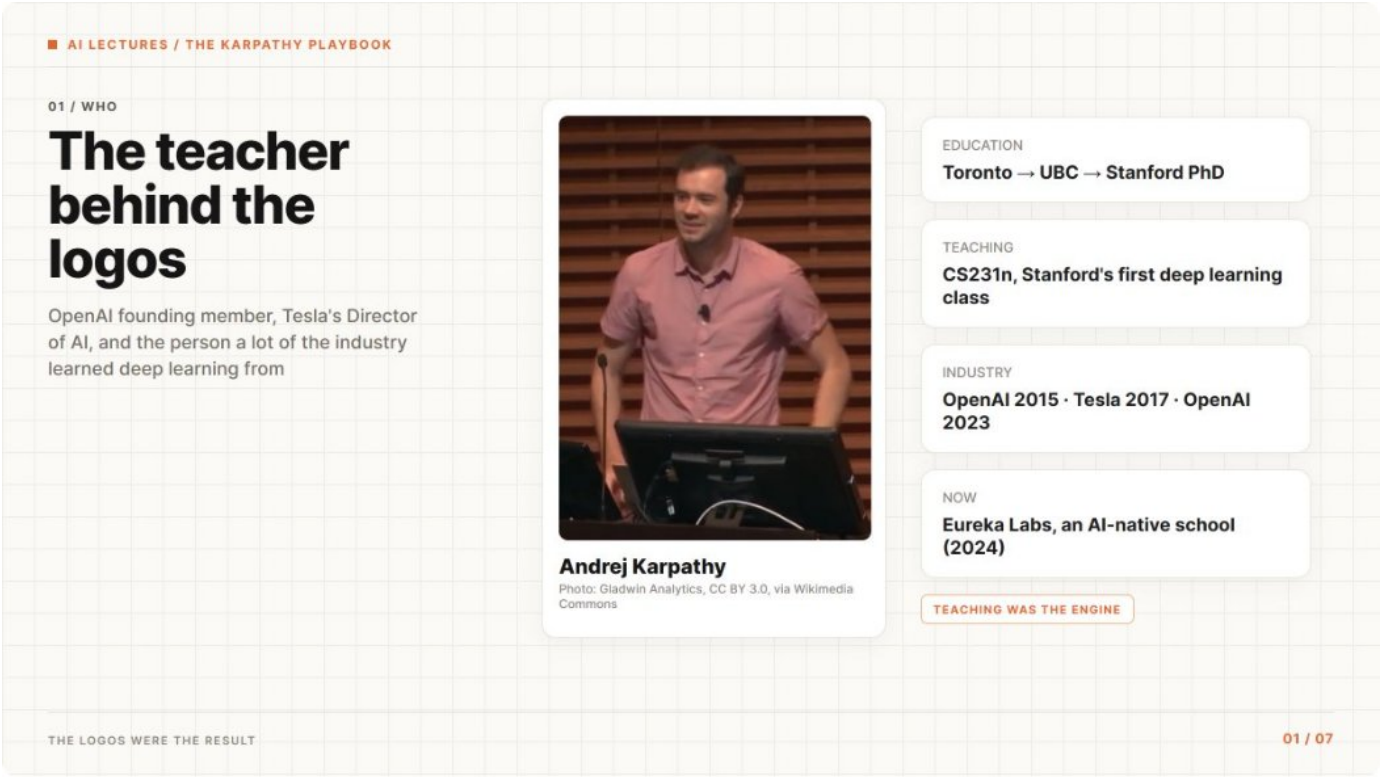


He held two of the most wanted jobs in AI. The thing that made him was a free class

Most people describe Andrej Karpathy as a list of logos. OpenAI founding member. Tesla's Director of AI. Stanford PhD. That list is true, but it hides the pattern

Look at what he actually does with his time. He takes something complicated, rebuilds it from scratch in as little code as possible, and explains it so clearly that a stranger can follow along

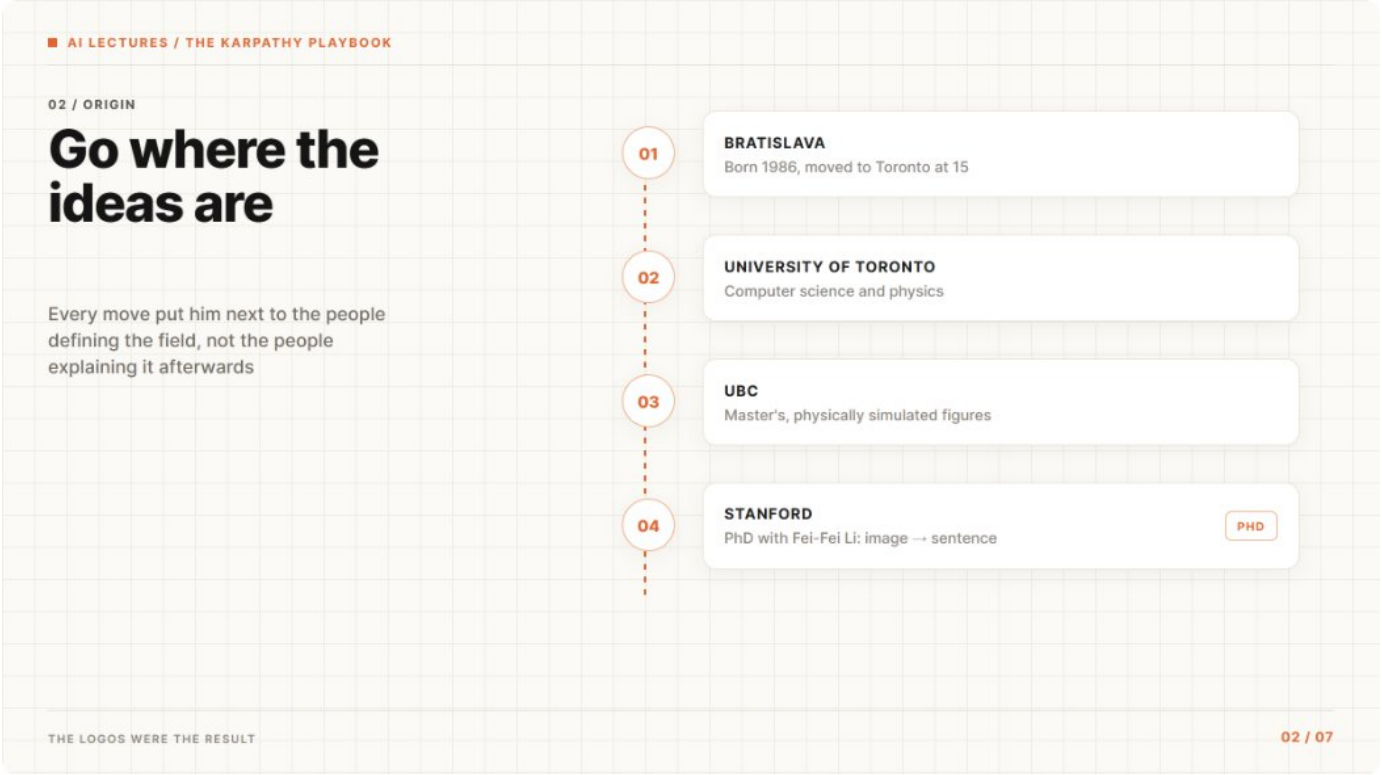
He did it as a PhD student. He did it after Tesla. He is still doing it now. The logos were the result. Teaching was the engine. This breakdown walks through that path one move at a time, and what you can copy from it this month



1. He Went Where the Ideas Were

Karpathy was born in Bratislava in 1986 and moved to Toronto at 15. He studied computer science and physics at the University of Toronto, the place where Geoffrey Hinton was teaching neural networks long before they were fashionable

Then came a master's at the University of British Columbia, working on controllers for physically simulated figures, and a PhD at Stanford under Fei-Fei Li. His PhD sat right on the line between vision and language: teaching networks to look at an image and describe it in a sentence

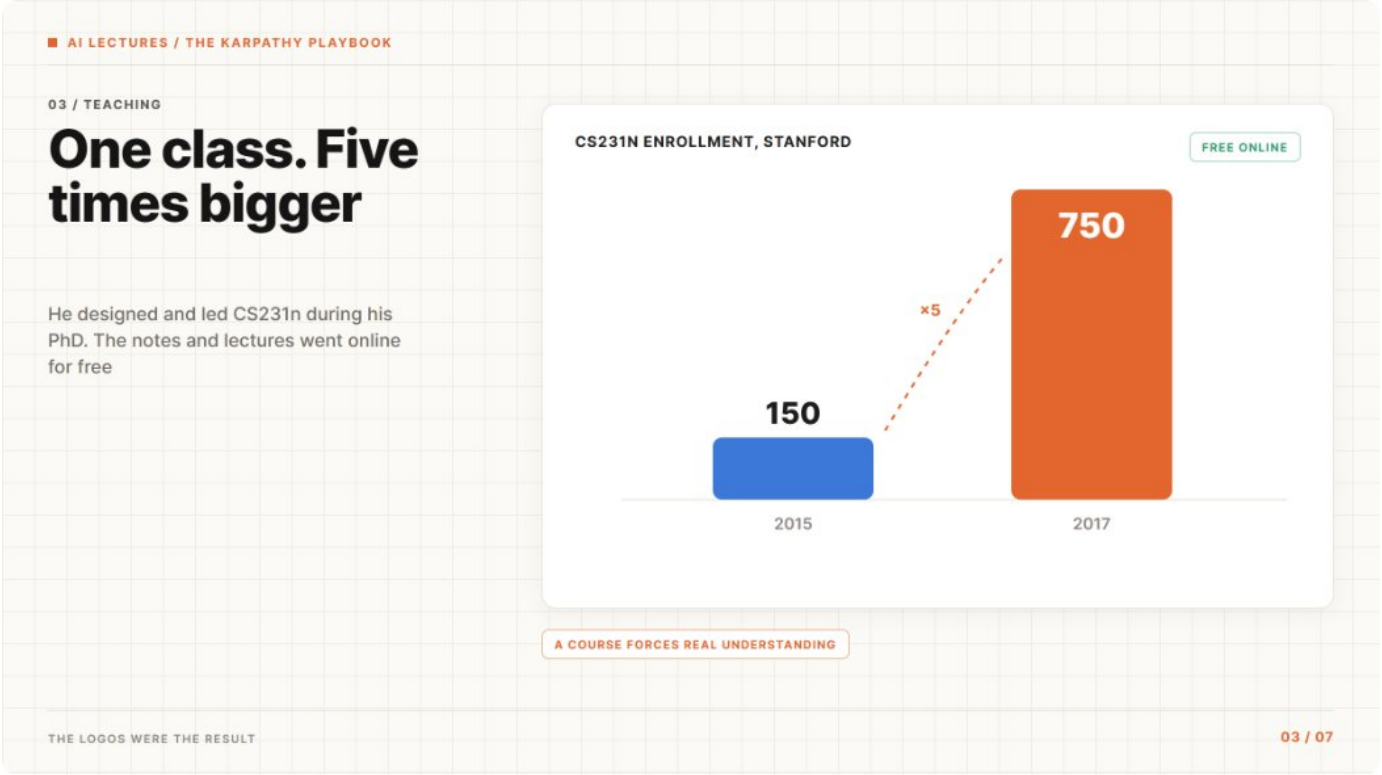


> **He kept putting himself next to the people who were defining the field, not the people explaining it after the fact**

---

2. He Built the Course He Wished Existed

During his PhD he designed and led CS231n with Fei-Fei Li, Stanford's first deep learning class. It grew from 150 students in 2015 to 750 in 2017



The classroom was the small part. The lecture videos and notes went online for free, and for years CS231n was how a big part of the industry learned convolutional networks

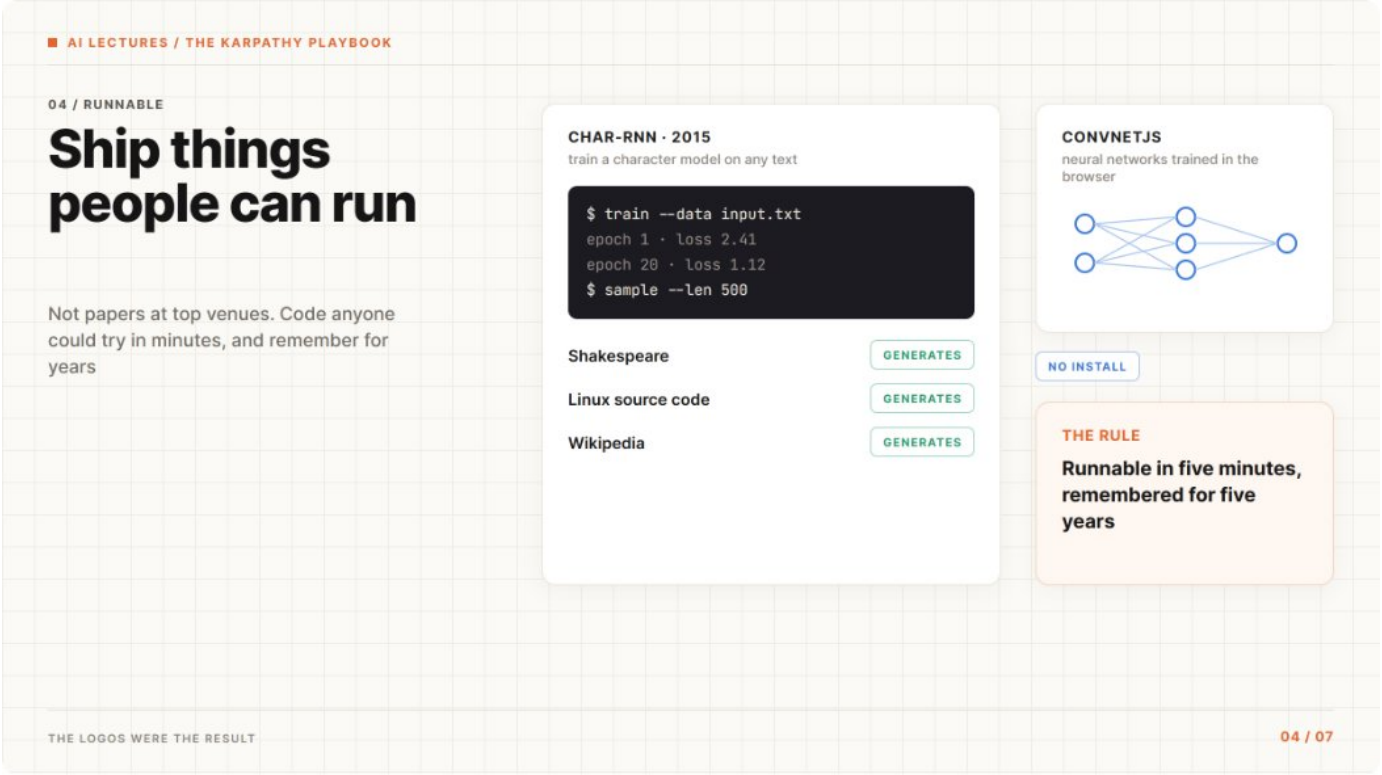
That is the first lesson worth stealing. A course is not a side project. It forces you to understand a topic so well that you cannot hide behind jargon

---

3. He Shipped Things People Could Run

In 2015 he published "The Unreasonable Effectiveness of Recurrent Neural Networks", a blog post with code that let anyone train a character-level model on their own text

People fed it Shakespeare, Linux source code and Wikipedia. It produced convincing fakes of all three, and recurrent networks suddenly felt real. He also built ConvNetJS, which trained neural networks right inside the browser



Neither one was a paper at a top venue. Both reached more people than most papers ever do

> If people can run it in five minutes, they remember it for five years

---

4. Founding Member, Then the Hardest Applied Job in AI

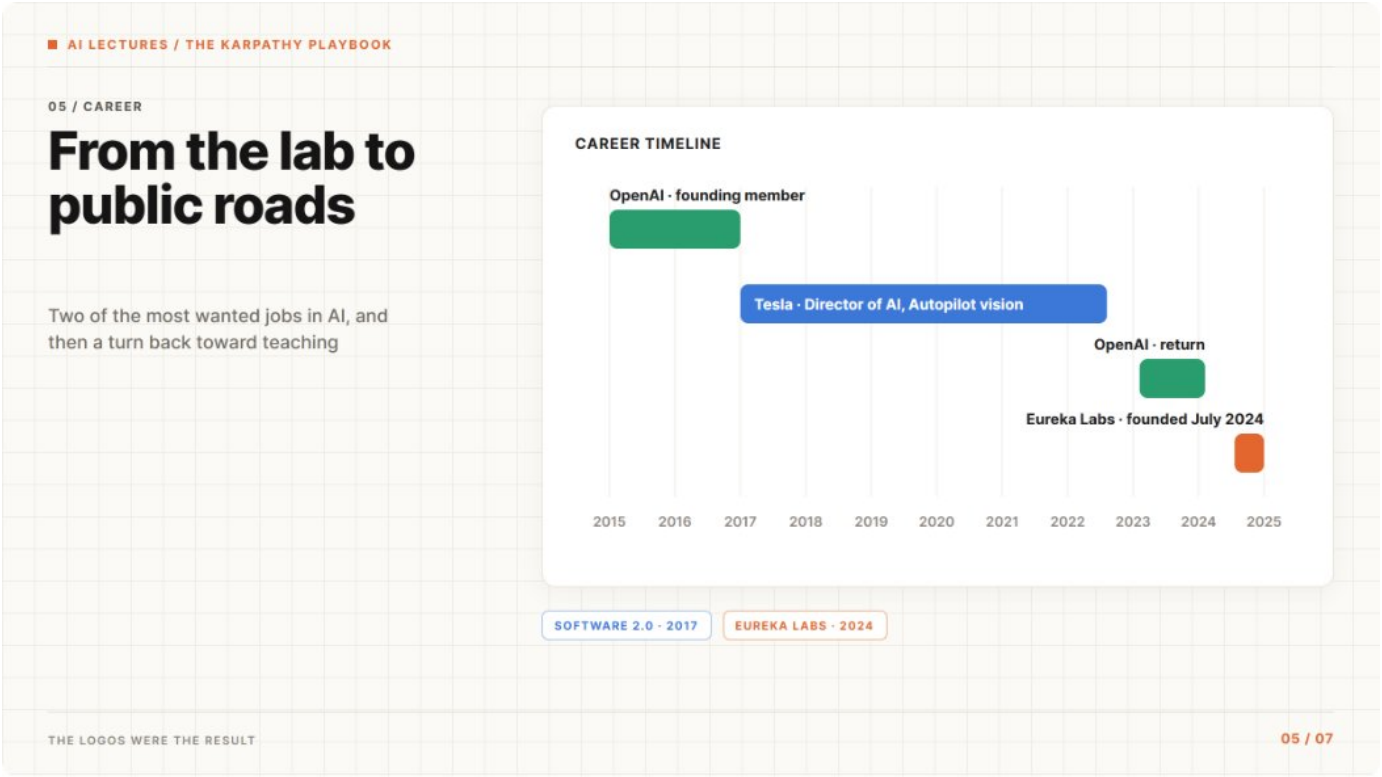
In 2015 he joined OpenAI as a research scientist and founding member

In 2017 he left to become Tesla's Director of AI, leading the computer vision work behind Autopilot and reporting to Elon Musk. That job was the opposite of a research lab. Real cars, messy real-world data, and a product that had to work on public roads

The same year he wrote "Software 2.0", an essay arguing that neural networks are a new kind of code. You stop writing the rules by hand and start curating the data that produces them

> The idea came from the job. Running a data engine at that scale is where it stopped being theory

He stayed for five years and announced his departure in July 2022



5. He Walked Away From the Titles

In February 2023 he returned to OpenAI. In February 2024 he left again

In July 2024 he announced Eureka Labs, an AI-native education company built around AI teaching assistants working next to human teachers

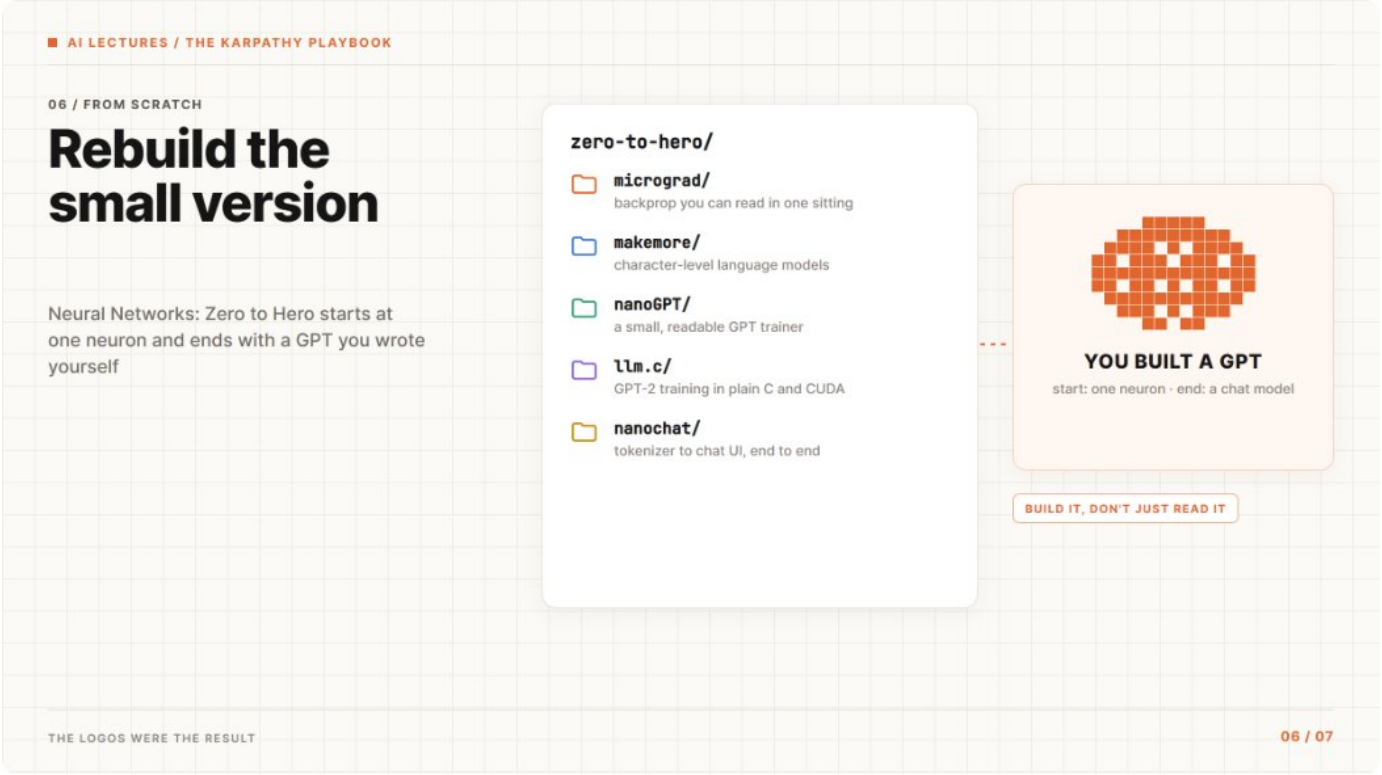
Most people climb toward bigger titles. He kept walking back toward teaching. That is not a detour. It is the same move he made at Stanford, with better tools

6. Zero to Hero: Rebuild Everything From Scratch

The strongest part of his public work is a YouTube series called Neural Networks: Zero to Hero

It starts with micrograd, a tiny autograd engine you can read in one sitting, and climbs step by step until you have built a GPT yourself. Along the way came the repos people still learn from:

Each one strips a famous system down to the part you actually need to understand



> The lesson is not "read his code". It is "build the small version yourself"

---

7. He Names the Shift Before Everyone Else

Karpathy keeps handing the industry its vocabulary

Software 2.0 in 2017

"Vibe coding" in February 2025: you describe what you want in plain language, accept the code the model writes, and steer by results instead of reading every line

Software 3.0 in his June 2025 talk at Y Combinator's AI Startup School, where he described LLMs as a new kind of computer that you program in English

His long lectures on how LLMs work, like Deep Dive into LLMs like ChatGPT and How I Use LLMs, became the default starting point for people entering the field

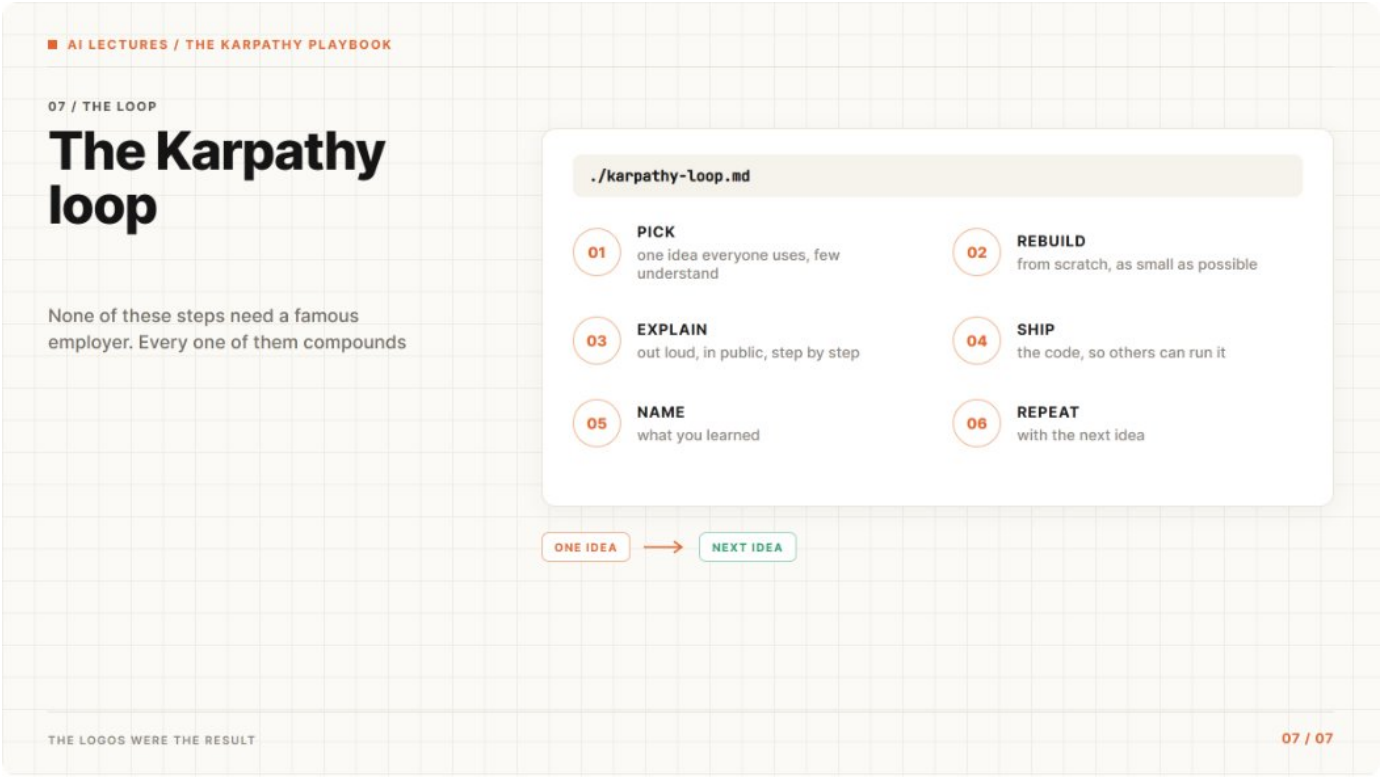
When you explain something well, you also get to name it

---

8. The Karpathy Loop

Put all of that together and the playbook is simple

None of these steps need a famous employer. Every one of them compounds



9. How to Run It This Month

You do not need to be Karpathy to use the loop. You need one topic and four weekends

Week 1. Pick one concept you use every day but cannot explain. Attention, embeddings, RAG, an agent loop

Week 2. Rebuild a toy version in the smallest code you can. Ugly is fine. Working is the point

Week 3. Record yourself explaining it from a blank screen. No slides

Week 4. Publish the repo and the video together, then write one post about the part that surprised you

The first one will be rough. The fifth one will be better than most paid courses

---

The Real Shift

AI changed what a career looks like

Credentials used to be the moat. Now a model can write the code, summarize the papers and draft the lecture notes

What it cannot do is decide what deserves to be understood, rebuild it with taste, and make it click for a room full of people

That is the part Karpathy has been doing for more than a decade

The logos came and went. The lectures are still there, and they keep compounding

---

If You Made It This Far

Pick one idea from this list and rebuild it this week

Which concept would you teach first?